從實體問題，到 AI 學習系統

A　國中｜Arduino 智慧遮雨棚

我負責 Arduino 接線、程式與 App Inventor 手機控制。這是我第一次把程式從螢幕上的運算，真正帶到感測器、馬達與實體裝置上，也因此開始接觸軟硬體整合。

2023 IEYI 臺灣選拔賽
國中組便利生活類｜**銀獎**

▶ 延伸資料｜發明展資料 ↗

B　高一｜FBref 球員數據分析

自學 Python、pandas 與爬蟲。單一球員成功後擴大請求，IP 被限制；理解 Rate Limiting 後加入請求間隔。**即使程式沒有 bug，不了解外部系統，整個系統仍可能失敗。**



▲ 球員近五場各指標比較

▶ 延伸資料｜GitHub 程式碼 ↗

C　高二｜自適應性輔助學習系統

▶ GitHub ↗　▶ Demo ↗

高二時我想要參加育秀盃，指導老師要我們先找身邊真正存在的問題。我問在高職教數學的父親，他說同一班學生程度差距很大，老師難以在同一堂課替不同學生安排不同進度的練習與回饋——這成為我開發「自適應輔助學習系統」的起點。

我的分工｜在三人團隊中，我主要負責教材進入系統、題目生成到學生作答回傳的資料主線，以及 Code-as-Content 的程式題生成；後續的 AST Healer 研究，也從這一部分延伸出來。

① 學生答完之後，下一步往哪走？

學生作答 → 作答紀錄 → 更新學習狀態 → **判斷下一步**

已掌握　→ 下一題／下一技能
尚未掌握　→ 找出弱點與前置概念 → 補強、AI 提示 → 推薦練習 → 再次作答 ↺

② Code-as-Content｜把「出題程式」當成題庫內容

題庫保存題型與教材內容，由語言模型產生 Python 出題程式；執行時帶入不同參數，同一支程式就能反覆產生不同題目與答案。

題型／教材 → **模型產生 Python 出題程式** → 帶入參數 → **題目＋答案**



學生端｜依章節與技能練習・AI 提示與補強・作答紀錄・後續推薦



教師端｜PDF／Word 教材匯入・班級管理・技能結構・查看學習紀錄

2026 扶輪盃全國高中 AI 競賽
高中組第一名　早期原型

→

第 23 屆育秀盃　系統完整成果
全國高中職 AI 應用類金獎

系統真正開始運作後，我開始把注意力放到另一個問題：程式能不能穩定、可驗證地運作。

系統維運問題，成為研究起點

A　研究起點｜大型模型的成本與延遲

Code-as-Content 每次出題都要呼叫大型模型，token、API 成本與延遲持續累積（Gemini API 10 月峰值約 1,560 元）。改用本地小模型後，卻發現它生成 Python 並不穩定。

怎麼讓小模型少做一點，卻做得更可靠？

B1｜生成前：哪些工作可事先抽離？

Scaffolding／Domain Functions

先整理工具、參數來源與呼叫順序，降低模型自己規劃的負擔。

B2｜生成後：哪些錯誤可用規則修？

AST Active Healer

只修固定規格能唯一判斷修法的局部錯誤。**不能確定修法，就不介入。**

C　初版驗證

41.00 Bare → **94.39** ＋Scaffolding → **97.41** ＋Full Healer

MCRI 為自建評估指標；97.41 是 MCRI 分數，不是通過率。三種模型都出現相同的改善趨勢。

D　研究歷程中的競賽、評審提問與轉折

校內科展 **優勝** → **第 66 屆東區科展** **優等** 未晉級全國 → **第 25 屆旺宏科學獎** **全國決賽 20 強** 全領域 846 件 → 入圍之後 收到評審建議

① 測試資料獨立嗎？
→ 開發／測試分離、評估前凍結規則

② Healer 修好了什麼？
→ 逐筆追蹤每個原始失敗

③ 自建 MCRI 夠當證據嗎？
→ 聚焦 Healer、改用 PASS／FAIL

F　Math16 重新驗證

▶ 旺宏決賽報告 ↗　▶ 評審建議 ↗

16 道公開會考題 × 3 模型 × 4 Prompt × 5 seeds ＝ 960 單元

4 題｜規則開發 240 單元 → **凍結 Prompt 與 Healer 規則** → **12 題｜保留題正式測試** 720 單元・不讀答案・不跨組

G　正式結果

720	342	42	9	2.63%
正式測試	原始失敗	Healer 修改	失敗→通過	**安全救援率**

- 2.63% 與初版 MCRI 97.41 是不同指標，不能直接比較。
- 342 個失敗中，144 個（約 42%）是答案或演算法錯誤，本就不在 Healer 範圍。

H　Healer 的邊界

能修｜輸出格式・局部結構・資料流・固定 API 規格可判定錯誤

不修｜數學答案・核心演算法・題意理解・需重新推理的錯誤

重新驗證後，我開始注意一套 AI 軟體系統要怎麼分開處理模型能力、流程設計、程式修復與測試邊界，也更確定想往軟體系統、程式分析與測試繼續學。